```
import pygame
from config import WIDTH, HEIGHT, font_terminal_small, C_TERMINAL
from state import STATE

class Console:
    def __init__(self):
        self.active = False
        self.input_text = ""
        self.history = []
        self.max_history = 50
        self.font = font_terminal_small
        self.bg_color = (0, 0, 0, 200)
        self.text_color = C_TERMINAL

    def toggle(self):
        self.active = not self.active
        self.input_text = ""

    def handle_event(self, event):
        if not self.active:
            return False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_BACKQUOTE:
                self.toggle()
                return True
            if event.key == pygame.K_RETURN:
                self.execute_command(self.input_text)
                self.input_text = ""
            elif event.key == pygame.K_BACKSPACE:
                self.input_text = self.input_text[:-1]
            elif event.unicode and event.unicode.isprintable():
                self.input_text += event.unicode
            return True
        return False

    def execute_command(self, cmd):
        cmd = cmd.strip()
        if not cmd:
            return
        self.history.append(f"> {cmd}")
        if len(self.history) > self.max_history:
            self.history.pop(0)

        parts = cmd.split()
        command = parts[0].lower()

        if command == "godmode":
            STATE.godmode = not getattr(STATE, "godmode", False)
            self.add_message(f"Godmode: {'ON' if STATE.godmode else 'OFF'}")

        elif command == "spawn_enemy":
            if STATE.player:
                px, py = STATE.player.rect.centerx, STATE.player.rect.centery
                try:
                    from enemy_ai import Enemy
                    e = Enemy(px + 100, py)
                    STATE.enemies.append(e)
                    self.add_message("Enemy spawned")
                except ImportError:
                    self.add_message("Cannot import Enemy")
            else:
                self.add_message("No player")

        elif command == "teleport":
            if len(parts) == 3 and STATE.player:
                try:
                    x, y = int(parts[1]), int(parts[2])
                    STATE.player.rect.centerx = x
                    STATE.player.rect.centery = y
                    self.add_message(f"Teleported to ({x}, {y})")
                except ValueError:
                    self.add_message("Usage: teleport X Y")
            else:
                self.add_message("Usage: teleport X Y")

        elif command == "give_ammo":
            wep = getattr(STATE, "current_weapon", "pistol")
            if wep == "uzi":
                STATE.uzi_ammo_in_mag = STATE.uzi_mag_size
                STATE.uzi_spare_mags = 10
            else:
                STATE.ammo_in_mag = STATE.mag_size
                STATE.spare_mags = 10
            self.add_message("Ammo refilled")

        elif command == "heal":
            if STATE.player:
                STATE.player.hp = getattr(STATE.player, "max_hp", 100)
                self.add_message("Player healed to full HP")

        elif command == "set_stage":
            if len(parts) == 2:
                stage = parts[1]
                mission = getattr(STATE, "active_mission", "[illegible]")
                if mission == "[illegible]":
                    try:
                        from [illegible] import [illegible]
                        stage_map = {
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                        }
                    except ImportError:
                        self.add_message("Cannot import mission stages")
                        return
                elif mission == "[illegible]":
                    try:
                        from [illegible] import [illegible]
                        stage_map = {
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                            "[illegible]": [illegible],
                        }
                    except ImportError:
                        self.add_message("Cannot import mission stages")
                        return
                else:
                    self.add_message(f"No stage support for {mission}")
                    return
                if stage in stage_map:
                    STATE.mission_stage = stage_map[stage]
                    if hasattr(STATE, "[illegible]_stage"):
                        STATE.[illegible]_stage = stage_map[stage]
                    if hasattr(STATE, "[illegible]_stage"):
                        STATE.[illegible]_stage = stage_map[stage]
                    self.add_message(f"Stage set to {stage}")
                else:
                    self.add_message(f"Unknown stage: {stage}")
            else:
                self.add_message("Usage: set_stage <stage>")

        elif command == "list":
            self.add_message("Commands: godmode, spawn_enemy, teleport X Y, give_ammo, heal, set_stage <stage>, list, help")

        elif command == "help":
            self.add_message("godmode - toggle invincibility")
            self.add_message("spawn_enemy - spawn enemy near player")
            self.add_message("teleport X Y - move player to coordinates")
            self.add_message("give_ammo - refill current weapon ammo")
            self.add_message("heal - restore player HP to max")
            self.add_message("set_stage <stage> - change mission stage")
            self.add_message("list - show all commands")

        else:
            self.add_message(f"Unknown: {command}. Type 'list' for commands.")

    def add_message(self, msg):
        self.history.append(f"  {msg}")

    def draw(self, surface):
        if not self.active:
            return
        console_h = 250
        bg = pygame.Surface((WIDTH, console_h), pygame.SRCALPHA)
        bg.fill(self.bg_color)
        surface.blit(bg, (0, HEIGHT - console_h))

        line_h = 18
        visible = (console_h - 30) // line_h
        start = max(0, len(self.history) - visible)
        y = HEIGHT - console_h + 10
        for line in self.history[start:]:
            txt = self.font.render(line, True, self.text_color)
            surface.blit(txt, (10, y))
            y += line_h

        prompt = self.font.render(f"> {self.input_text}", True, self.text_color)
        surface.blit(prompt, (10, y))
        if pygame.time.get_ticks() % 1000 < 500:
            cursor = self.font.render("_", True, self.text_color)
            surface.blit(cursor, (10 + prompt.get_width(), y))
```